

SPCA: Stream Parser Confusion Attack for Web Application Firewall Evasion in HTTP/2

Kyungrok Choi*

Korea University

Computer Science and Engineering

Seoul, Republic of Korea

krchoi@isslab.korea.ac.kr

Woonghee Lee*

Korea University

Computer Science and Engineering

Seoul, Republic of Korea

whlee@isslab.korea.ac.kr

Junbeom Hur

Korea University

Computer Science and Engineering

Seoul, Republic of Korea

jbhur@korea.ac.kr

Abstract

Web Application Firewalls (WAFs) are widely deployed as a primary defense mechanism against injection-based web attacks by inspecting HTTP traffic for malicious patterns. However, structural inconsistencies in HTTP/2 stream parsing introduce a protocol-level attack surface that remains insufficiently examined. We propose the *Stream Parser Confusion Attack* (SPCA), a novel evasion technique that exploits discrepancies between WAFs and backend HTTP/2 servers in processing stream dependencies and priorities. SPCA operates without altering payload content, relying solely on RFC-compliant manipulation of stream priority weights and dependency trees to deliver unmodified malicious inputs past WAF inspection. To evaluate the feasibility and generality of SPCA, we design three well-defined stream topologies—skewed, k-ary, and unbalanced—each capturing unique structural traits observed in real-world HTTP/2 scheduling patterns. Each topology’s dataset consists of 500 structurally distinct requests, derived by embedding 100 malicious test cases across five distinct priority levels. We transmit these requests against 13 commercial and open-source WAFs and 20 backend web frameworks in a black-box setting. Each topology individually achieves a bypass success rate of 49.66% for the skewed tree, 44.62% for the k-ary tree, and 46.38% for the unbalanced tree. Under the concurrent attack with three topologies, the overall success rate exceeds 89% on average against the open-source and commercial WAFs. These findings demonstrate that structure-only protocol-compliant manipulation is sufficient to systematically bypass modern WAFs, revealing critical blind spots in HTTP/2-aware traffic inspection. We responsibly disclosed the identified issues to all affected vendors and received acknowledgments of the disclosures.

CCS Concepts

• Security and privacy → Web application security.

Keywords

Web Application Firewall; Evasion Technique; Protocol Parsing Discrepancy

*Equally contributed.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792517>

ACM Reference Format:

Kyungrok Choi, Woonghee Lee, and Junbeom Hur. 2026. SPCA: Stream Parser Confusion Attack for Web Application Firewall Evasion in HTTP/2. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792517>

1 INTRODUCTION

Web Application Firewalls (WAFs) serve as critical defenses for modern web services, acting as intermediaries that inspect inbound HTTP traffic to prevent web attacks such as SQL injection (SQLi), cross-site scripting (XSS), and command injection. They enable application-layer protection without modifying backend code and have become foundational in cloud platforms and enterprise infrastructure due to regulatory mandates such as PCI-DSS [34].

Recent measurements show steady growth of HTTP/2 traffic, accounting for more than half of all web traffic on the Internet since 2023 (e.g., 70% in 2023) [22, 28, 40]. This steady growth underscores the increasing dominance of HTTP/2 in the real world, particularly due to its multiplexed stream model and enhanced efficiency. As HTTP/2 becomes the default transport layer in modern content delivery platforms, its protocol-level semantics such as stream dependencies, frame priority, and HEADERS-DATA separation form an increasingly critical attack surface. This trend highlights the urgent need for systematic assessment of current web security mechanisms including Web Application Firewalls (WAFs) in the context of HTTP/2 structure rather than legacy behavior of HTTP/1.1.

To circumvent WAF defenses, previous studies have focused predominantly on content-level evasion strategies [3, 4, 18]. These include payload mutation techniques such as encoding tricks [11], adversarial transformations [41], and obfuscation using machine-learning [3], aiming to deceive shallow pattern matching filters. However, as WAFs incorporate more semantic-aware inspection and taint tracking techniques [8], the effectiveness of such methods is steadily declining. In contrast, a more potent class of attacks has brought about protocol-level evasions that exploit structural discrepancies between how WAFs and backend web frameworks parse and reassemble HTTP/2 streams. The discrepancy lies in the fact that WAFs often perform shallow, stateless inspection, while backend frameworks reassemble full requests by parsing and reassembling HEADERS and DATA frames based on the HTTP/2 protocol. This asymmetry in parsing logic creates a critical attack surface: attackers can deliver malicious DATA that bypasses WAF detection but is later executed after full reassembly on the server. Despite their severity, most existing tools are ill-suited to discover such flaws for three reasons: (1) they focus on payload fuzzing rather

than manipulating protocol structure, (2) black-box testing against commercial WAFs lacks observability into backend behavior, (3) there is no systematic method to test stream-level semantics such as HTTP/2 priority, dependency, and multiplexing key mechanisms that affect request reassembly. Thus, protocol-level evasion vulnerabilities remain underexplored.

In this paper, we propose *Stream Parser Confusion Attack (SPCA)*, a novel protocol-level evasion technique that leverages HTTP/2 stream semantics to mislead WAF parsers. Unlike content mutation, SPCA does not alter the payload. Instead, it rearranges the stream dependency structure and transmission schedule using RFC-compliant [39] configurations to confuse WAF parsing logic while ensuring that the backend server still reassembles and executes the complete malicious request. SPCA targets parsing discrepancies arising from HTTP/2 features such as HEADERS, DATA separation, stream priority scheduling, and multiplexed dependency trees.

To validate SPCA, we design three well-defined stream tree topologies, each reflecting a distinct configuration of stream dependencies and priority weights for evasion strategy: *skewed trees* test for depth-based stream reordering, *k-ary trees* evaluate sibling stream concurrency, and *unbalanced trees* stress asymmetrical scheduling paths. For each topology, we embed 100 malicious test cases across five distinct priority weights, generating 500 structurally valid HTTP/2 requests. These requests are evaluated against 13 commercial and open-source WAFs and 20 widely used HTTP/2-compliant web frameworks, enabling comprehensive measurement of bypass success and backend execution. We evaluate end-to-end bypass success rates of each topology both individually and concurrently by verifying whether backend frameworks execute the reassembled malicious requests. For individual attacks, each topology shows bypass rates of 49.66% (skewed), 44.62% (k-ary), and 46.38% (unbalanced). Whereas, for concurrent attack with three topologies where we generated and transmitted three structurally distinct HTTP/2 requests per input, a bypass success rate exceeding 84% is achieved across all test cases, revealing that even modern WAFs including Cloudflare, AWS WAF, and Fortinet fail to block SPCA attacks in practice. Our findings underscore the urgency of structure-aware inspection and RFC-conform parsing in future WAF architectures. Our key contributions of this work are as follows:

- **New Protocol-level Evasion Technique.** We introduce SPCA, a novel RFC-compliant evasion method that manipulates HTTP/2 stream structure and scheduling to bypass WAFs without modifying payload content.
- **Systematic Tree-based Evaluation Methodology.** We design three stream dependency trees (skewed, k-ary, and unbalanced), each targeting a unique WAF parsing failure mode and reflecting prioritization patterns seen in real-world deployments.
- **Topology-Aware Evasion Strategy.** We demonstrate that each stream topology enables bypass of distinct test cases, contributing to overall coverage. According to our experiment, it achieves individual bypass rates of 44.62% in k-ary tree, 49.66% in skewed tree, 46.38% in unbalanced tree on average across 13 WAFs; and 89% under the concurrent attack.
- **End-to-End Backend Execution Validation.** We confirm that SPCA not only bypasses WAF inspection but also leads to real backend execution across 20 widely used HTTP/2-compliant

frameworks. This end-to-end validation goes beyond traditional detection-based evaluation by verifying the execution impact of evasive requests at the application layer.

- **Responsible Disclosure** We responsibly reported our findings to affected vendors and actively engaged with each security team. Especially, AWS VDP team is in the process of creating the appropriate remediation.

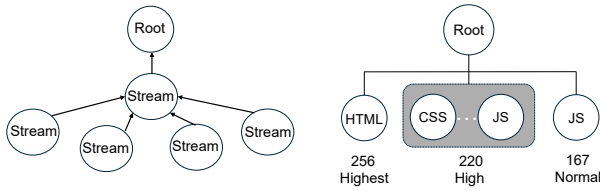
2 BACKGROUND

2.1 HTTP/2 Multiplexing & Prioritization

HTTP/2 Multiplexing. HTTP/2 is a binary protocol that enables multiple HTTP requests and responses to be transmitted concurrently over a single TCP connection [39]. Unlike HTTP/1.1, which typically opens a new connection for each request, HTTP/2 achieves concurrency by dividing the connection into multiple independent streams, a mechanism known as multiplexing. Each stream has a unique integer identifier and carries a sequence of protocol-defined frames. Figure 1 illustrates this architectural difference. In HTTP/1.1, clients must establish multiple parallel TCP connections to fetch resources such as scripts and images, resulting in connection overhead and inefficient resource utilization. In contrast, in HTTP/2, all resources are transmitted over a single connection using stream multiplexing. Each stream of multiplexed streams operates independently, allowing interleaved frame transmission across the connection.

HTTP/2 Stream Prioritization. Streams in HTTP/2 transmit two primary types of frames: HEADERS frames, which contain request metadata such as method and path, and DATA frames, which carry the message body [17, 22, 23, 29, 38, 42]. Since frames of different streams can be interleaved on the wire, recipients buffer frames per stream (by stream ID) and reassemble each request in-order within that same stream. Therefore, any system processing HTTP/2 traffic must track stream boundaries and frame types to maintain correct association between HEADERS and DATA [24].

In addition to reassembly, HTTP/2 introduces a stream prioritization mechanism that influences how frames are scheduled and transmitted. Each stream may declare a dependency on another stream and assign it a weight value from 1 to 256, forming a hierarchical tree structure, as shown in Figure 1. Figure 1(a) illustrates a generic HTTP/2 stream dependency tree, where multiple streams are organized into a hierarchical structure using parent-child relationships and associated weights. Building on this conceptual structure, Figure 1(b) provides a real-world example in which web resources—HTML, CSS, JavaScript, and images—are each assigned specific weight values to control their transmission order. The weight values directly influence how the HTTP/2 scheduler allocates transmission opportunities among active streams. Under bandwidth constraints, streams with higher weights are selected more frequently for frame transmission, resulting in earlier delivery of critical resources such as HTML and CSS. In contrast, lower-weight streams—typically associated with non-blocking assets like images or fonts—are scheduled less often and delivered later in the connection timeline. The hierarchical structure shown in Figure 1(a) demonstrates how both dependency links and stream weights jointly determine transmission order.



(a) Example of hierarchical dependencies. (b) Priority-based transmission order of web resources.

Figure 1: HTTP/2 Dependency Trees.

2.2 Web Application Firewall

A Web Application Firewall (WAF) is a fundamental component of modern web security architecture [34]. Deployed at the application layer, a WAF acts as an intermediary between clients and web applications, inspecting incoming HTTP traffic to detect and block potentially malicious requests [11].

WAF Workflow. Most WAFs implement a rule-based inspection pipeline consisting of three phases: parameter parsing, pattern matching, and action execution [10, 11, 25, 43]. In the first phase, the WAF extracts structured parameters from the incoming HTTP request. In the second phase, the WAF evaluates the parsed parameters against a detection ruleset based on OWASP Core Rule Set (CRS). The OWASP CRS [27] is the most widely adopted detection framework in both open-source and commercial WAF products, offering standardized protection against common web vulnerabilities including those in the OWASP Top Ten. Each rule in CRS targets specific attack signatures—such as SQL keywords or scripting payloads—expressed as regular expressions, fixed token sequences, or heuristic patterns. In the final phase, if a rule match is found, the WAF enforces a predefined response such as rejecting the request, logging the event, or triggering an alert. These phases in WAF are performed prior to backend processing.

HTTP/1.1 vs. HTTP/2 in WAFs. WAFs assume that each HTTP request arrives as a complete and ordered unit. Under HTTP/1.1, the assumption generally holds: requests are transmitted sequentially over dedicated TCP connections, allowing the WAF to inspect them atomically. However, as described in Section 2.1, HTTP/2 replaces the model with a multiplexed, frame-based transport mechanism. Each request is fragmented into HEADERS and DATA frames, which may arrive out of order across interleaved streams. To apply rules correctly under the model for HTTP/2, a WAF must maintain stream-level state, buffer incoming frames, and reassemble complete logical requests prior to inspection. In practice, however, many WAFs inspect individual frames or apply heuristics based on the arrival order, prioritizing throughput over complete request reassembly [1, 6, 10]. This leads to the exploitable parsing discrepancy: the attacker constructs a protocol-compliant HTTP/2 request in which fragmented frames are intentionally scheduled to evade WAF inspection and later correctly reassembled by the backend.

3 THREAT MODEL & CHALLENGES

3.1 Threat Model

We consider a threat model where an attacker leverages structural ambiguities in HTTP/2 to bypass WAFs. In this setting, all

inbound traffic is routed through a WAF, inspecting HTTP/2 requests and enforcing security policies such as the OWASP Core Rule Set (CRS) by primarily analyzing selected frame types, notably HEADERS [19]. As discussed in Section 2.2, many WAFs rely on simplified parsing strategies failing to interpret stream dependencies or associate frames across multiple streams. The backend application, in contrast, is assumed to implement a fully RFC 9113-compliant HTTP/2 parser [39] which reassembles all incoming frames—including HEADERS, DATA, and other control frames—based on stream identifiers and dependency structures.

The attacker is a remote client without privileged access to the WAF or the backend system. Interacting solely over standard HTTP/2 connections, the attacker can initiate multiple streams within a single connection and manipulate stream dependencies via PRIORITY frames, thereby controlling the order and timing of frame delivery. Notably, the attack does not rely on any transport-level or implementation-specific vulnerabilities. Instead, the goal of the attack is to deliver a malicious payload to the backend without detection.

The attacker sends multiple streams, each composed of a header frame and associated data frames. In one selected stream, the header frame remains unaltered, while the attacker injects a malicious request into the data frame of the same stream [20]. Both frames individually comply with the HTTP/2 specification, and the overall structure avoids any content obfuscation [21]. The success of our attack relies on a parsing gap between WAFs and backend servers. The WAF tends to inspect only the HEADERS frame and deems the request benign, while the backend reassembles both HEADERS and DATA frames of the same stream into a single logical request, ultimately executing the payload. This divergence in interpretation exposes a parsing discrepancy in protocol-layer processing that attackers can systematically exploit by leveraging HTTP/2’s structural flexibility.

3.2 Key Challenges

Unlike conventional evasion strategies that rely on content obfuscation, our attack manipulates protocol-level HTTP/2 stream structure—such as priority weights and dependency hierarchies—without altering payload content. To examine the feasibility of structure-only evasion, we begin with a preliminary analysis and draw two key challenges that emerge from the findings.

Preliminary Analysis: Can Priority Manipulation Alone Induce WAF Evasion?

Our goal of the preliminary analysis was to examine whether priority-based scheduling alone, without any dependency manipulation, can affect WAF detection behavior, thereby isolating the influence of prioritization from structural complexity and establishing a minimal yet informative baseline. We selected 100 malicious HTTP/1.1 requests that had been reliably blocked by a diverse set of 13 target WAFs, including both commercial and open-source implementations. The 13 WAFs were chosen based on market share, documented HTTP/2 support, and prior research coverage [34, 43]. All malicious requests were repackaged into HTTP/2 format and distributed across 20 logically independent streams using the h2 Python library [7]. Unlike higher-level HTTP/2 clients or command-line tools (e.g., curl or nghttp2), the h2 library exposes low-level

protocol interfaces that allow explicit construction of HEADERS and DATA frames, manual control over stream IDs, and precise configuration of dependency trees and priority weights. By leveraging the functionalities, we could simulate attacker-controlled stream structures in a fully RFC-compliant manner while preserving payload integrity. Therefore, we set up and tested two experimental cases:

- **Uniform Priority:** The library's implicit stream creation behavior was preserved. All streams assigned uniform priority, and no explicit scheduling or dependency logic was applied.
- **Priority Manipulation:** The library's default scheduling was disabled. The 20 streams were partitioned into five groups, each containing four streams. The stream IDs were assigned sequentially, and priority weights were applied in ascending order such that lower stream IDs carried lower weights. As shown in Table A1, each stream was manually assigned one of five priority weight groups based on empirical distributions observed in live HTTP/2 traffic [41]. Notably, no stream dependencies (e.g., parent-child relationship) were declared in either configuration.

Our setups enabled a controlled comparison of WAF parsing behavior across multiple priority levels while holding all other request parameters constant. As shown in Table A4 in Appendix, the priority manipulation achieved higher evasion rates in all target WAFs, compared to the uniform priority. However, the results also reveal important limitations of priority manipulation alone. For certain WAFs (e.g., Safeline WAF in Table A4 in Appendix), the increase in evasion rate was only marginal. More notably, as shown in Table A1, priority manipulation sometimes even resulted in lower evasion rates compared to uniform priority, particularly for specific priority groups (e.g., **Lowest** in Table A1 in Appendix). It implies that manipulating stream priorities in isolation does not guarantee consistently high evasion rates across different WAF implementations, highlighting the need for additional structural or dependency-based strategies to achieve robust evasions. This motivates the first core challenge of our study.

Challenge 1: Can RFC-compliant dependency trees improve WAF evasion capability?

The first challenge addresses whether manipulating stream dependency trees, without violating RFC compliance, enables more efficient and targeted evasion than solely depending on priority manipulation. Unlike flat stream prioritization, tree-structured layouts encode explicit parent-child relationships, potentially imposing additional constraints on frame scheduling and interpretation. To examine the effect of such structural variations of stream dependency trees, we set out to design tree topologies that best highlight their impact on WAF evasion. In developing the tree topologies, it was essential to ensure compliance to RFC 9113 [39]—preserving acyclic graph structures, valid priority weights, and correct parent-child assignments—so that the resulting trees would evade WAF detection while remaining fully functional in RFC-compliant web applications. Note that, by ensuring full compliance, we aim to provoke differential parsing behaviors that arise not from malformed syntax but from parsing discrepancy gaps.

Challenge 2: Can Protocol-Level Evasion Lead to Web Application Execution?

While WAF evasion is a necessary condition for successful attacks, it is not sufficient unless the backend application also reassembles and processes the evaded request. This challenge investigates whether protocol-level evasion actually results in meaningful execution within the backend logic. To evaluate whether protocol-level evasion leads to backend execution, we examine how backend web frameworks reassemble and handle reassembled requests at the application layer. Specifically, we configure each framework with predefined routes and instrument custom logging to verify whether evaded HTTP/2 requests—after backend reassembly—will result in observable execution behaviors.

4 METHODOLOGY

4.1 Stream Parser Confusion Attack Workflow

Figure 3 presents the complete workflow of *Stream Parser Confusion Attack* (SPCA), which is divided into eight sequential stages designed to evaluate protocol-level evasion against WAFs through HTTP/2 stream manipulation. Each stage numbered in the figure corresponds directly to a specific component in our attack pipeline.

- (1) **Establish HTTP/2 Connection.** The attack begins by establishing a single HTTP/2 TCP connection to the target server. The connection supports multiplexed streams, enabling concurrent transmission of HEADERS and DATA frames with distinct dependencies and priorities.
- (2) **Construct Stream Tree Topology.** SPCA generates a stream dependency tree following one of three well-defined topologies—skewed, k-ary, or unbalanced, as described in Section 4.2.
- (3) **Manipulate Stream Priority.** Using the constructed trees, SPCA sends multiple streams, each composed of a header frame and associated data frames. In one selected stream, the HEADERS frame remains unaltered, while SPCA injects a malicious request into the DATA frame of the same stream. To control the delivery order, distinct weight values ranging from 1 to 256 are assigned to streams. We assign higher weights to competing benign streams while preserving the ordering policy in RFC 9113 in order to create a temporal gap that some WAFs fail to bridge during inspection.
- (4) **WAF Evasion Attempt.** The full request is sent to a WAF (e.g., AWS WAF, Azure) without any internal instrumentation.
- (5) **WebApp Reassembly and Execution Validation.** To confirm successful execution, we monitor for backend behaviors, such as reflected payloads, error responses, or new log entries, that indicate application-layer processing of the request.
- (6) **Result Collection.** The response logger records outcomes such as HTTP status codes (e.g., 403 for block), backend logs, and payloads. These results are used to determine whether the request bypassed the WAF and triggered backend execution.

4.2 Design of Tree Topologies & Priority Manipulation

We construct three well-defined stream dependency trees, each designed to isolate a specific structural property of HTTP/2 scheduling in terms of vertical dependency depth, horizontal concurrency, and hierarchical irregularity: *skewed*, *k-ary*, and *unbalanced trees* (All kinds of trees are shown in Figure 2). These topologies are

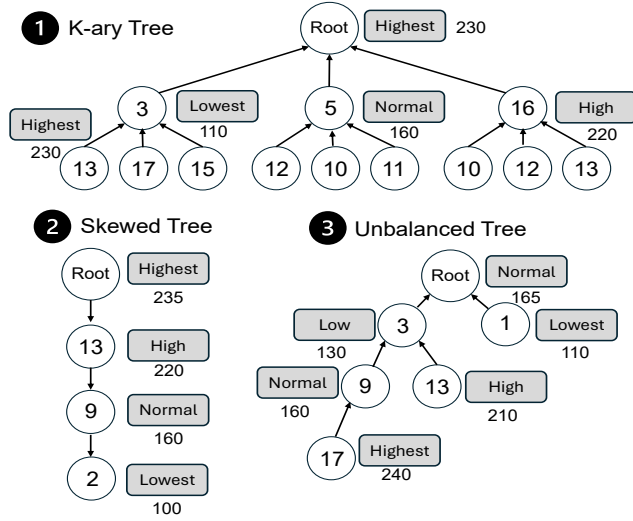


Figure 2: Tree Topologies Used in SPCA

grounded in real-world browser traffic patterns and RFC-compliant HTTP/2 prioritization semantics [42]. Note that, the malicious request is carried entirely within a single target stream, especially in DATA frame while HEADERS frame is unaltered.

1 K-ary Tree: Inducing Sibling-Level Concurrency. In the k-ary tree, a parent stream branches into three or more children with equal depth but independent execution paths. We inject the malicious payload in the DATA frame of the stream selected, and manipulate the sibling streams' weights to reverse their transmission sequence. The tree tests whether WAFs can semantically associate control and payload frames that share proximity but not lineage, mimicking real-world prioritization graphs under multiplexed browser requests.

2 Skewed Tree: Modeling Vertical Dependency Chains. The skewed tree simulates deeply nested stream chains commonly produced by browser prefetching and recursive dependency settings. Each stream is a direct child of its predecessor, forming a vertical chain. We select the malicious stream and introduce dummy benign streams to increase path length and contention. By assigning a lower priority to the malicious stream and higher priorities to benign streams, the priority manipulation can create a temporal inspection window. Note that the scheduling is RFC-compliant.

3 Unbalanced Tree: Inducing Asymmetric Stream Hierarchies. The unbalanced tree introduces non-uniform branching and irregular depth, simulating the reprioritization artifacts often produced by CDNs and adaptive HTTP/2 clients [42]. In contrast to the balanced sibling layout of the k-ary tree, the unbalanced tree can show disparity in stream lineage and weight hierarchy. Unlike the skewed tree's linear chain, the unbalanced topology integrates both shallow and deep branches within a single hierarchy. Some branches terminate at shallow depths, while others extend much deeper, creating asymmetry in both stream lineage and priority distribution. The malicious stream is selected on a deeper subtree within this hierarchy, where contention with high-weight sibling branches introduces scheduling asymmetry.

RFC Compliance in a Real-World. All tree configurations are strictly compliant with HTTP/2 dependency semantics as defined in

RFC 9113 [39]. These tree designs not only serve as diverse test cases for WAF parsing robustness, but also generalize to the prioritization strategies observed in major production deployments, including reprioritization traces at the browser and CDN level [21, 42].

4.3 WebApp Request Reassembly

The reassembly behavior of backend web servers constitutes the semantic foundation that enables SPCA. When receiving interleaved HTTP/2 frames from multiple prioritized streams, modern backend implementations such as NGINX, and Node.js reassemble each HTTP request on a per-stream basis using stream identifiers, frame types, and the HTTP/2 state machine defined in RFC 9113 [39]. Although these frames may arrive out of conventional order due to stream prioritization, the backend parser buffers incomplete transactions and reconstructs the request before invoking application-layer logic. In contrast, WAFs often apply stateless, per-stream parsing logic that lacks cross-stream correlation.

4.4 WebApp Execution Validation

To observe the downstream effects of protocol-level evasion, we instrument each backend web application to expose execution traces upon receiving and processing HTTP/2 requests. As shown in Table A2 in Appendix, our testbed includes 20 widely used frameworks across multiple languages, including Python (*Flask*, *FastAPI*), PHP (*Laravel*, *Symfony*), Node.js (*Express*, *Koa*), Java (*Spring Boot*), and Go (*Gin*, *Beego*). When a WAF forwarded request reaches the backend and is reassembled successfully, the embedded payload such as a SQL injection or XSS are passed to framework-specific request handlers (e.g., form parsers or query processors). The handlers may trigger application logic that reflects the payload in the response body, logs an exception, or causes observable side effects. To minimize architectural differences between frameworks and ensure consistent execution validation, we adopted the following methodology:

- **SPCA Injection Entry Points.** Each framework was configured to receive SPCA requests through a predefined route, such as `/search?id=...`, or a structured POST form endpoint. The injected payloads followed a fixed pattern designed to trigger SQL parsing errors or bypass logic in the backend.
- **Log Collection and Request Mapping.** Moreover, to collect detailed framework-level parsing logic that is not exposed through default logging, we instrumented handlers to explicitly record handling of malicious test case and request processing.
- **Execution Confirmation Criteria.** A request was considered *executed* if at least one of the following conditions was met: (1) the payload input was reflected in the HTTP response, (2) the backend raised an SQL-related exception (e.g., *SQLSTATE*, *QueryException*), or (3) a custom handler log was recorded indicating parameter parsing.

This approach enabled us to go beyond verifying WAF bypass and to systematically determine whether backend application logic was triggered by the SPCA request.

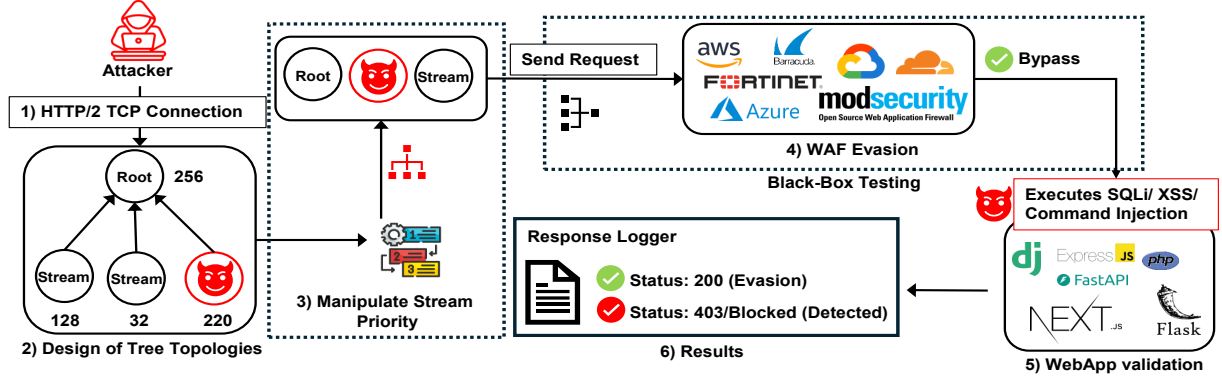


Figure 3: SPCA Experiment Overview.

5 EVALUATION

5.1 Experimental Setup

To ensure experimental reproducibility and isolation, we conduct all experiments within a fully containerized Docker environment. Each test executes in an isolated Linux network namespace to prevent cross-contamination between trials. Our experimental architecture deploys target WAFs and backend web frameworks in independent containers, with SPCA requests generated from a dedicated sender container. We maintain consistency across all trials through centralized orchestration using automated Python scripts that handle request dispatch, logging, and backend instrumentation.

Building upon this infrastructure, we evaluate 13 WAFs representing both commercial and open-source implementations. Our selection criteria are based on current market share reports and verified HTTP/2 protocol support [26]. To validate end-to-end attack effectiveness, we deploy 20 HTTP/2-compliant backend web frameworks spanning eight programming languages. Complete web framework specifications are provided in Table A2 in Appendix, with each application configured to record parsed HTTP/2 fields and maintain comprehensive backend execution traces. Our SPCA client implementation utilizes Python 3.9 with the h2 library [7] to achieve fine-grained control over HTTP/2 stream construction. 100 malicious test cases, which were used during our preliminary analysis (Section 3.2), were tested to compare the evasion success rate of priority manipulation only and SPCA. For systematic evaluation of priority-based evasion, we use five structural variants, which were also used in the preliminary analysis (Section 3.2), of each payload based on distinct priority assignments, yielding 500 SPCA requests per dependency topology. Our priority levels follow empirically derived ranges that reflect realistic HTTP/2 traffic patterns:

- **Lowest Priority:** Stream ID 1–4, Weight 100–120
- **Low Priority:** Stream ID 5–8, Weight 130–150
- **Normal Priority:** Stream ID 9–12, Weight 160–190
- **High Priority:** Stream ID 13–16, Weight 200–220
- **Highest Priority:** Stream ID 17–20, Weight 230–256

Finally, we evaluate three canonical dependency topologies: k-ary, skewed, and unbalanced. To sum up, our complete experimental matrix consists of 500 structurally distinct requests per topology, resulting in 1,500 SPCA trials per WAF (500 requests × 3 topologies). Our systematic approach enables rigorous analysis of how stream

dependency structures and priority scheduling jointly influence WAF bypass effectiveness across diverse implementations.

Evaluation Metrics We evaluate SPCA effectiveness using three quantitative metrics.

- **Evasion Success Rate (ESR):** This metric quantifies the proportion of HTTP/2 requests that successfully evade WAF detection under structurally manipulated conditions. For each WAF, we issue 500 requests per topology—*k*-ary, *skewed*, and *unbalanced* resulting in 1,500 test attempts per WAF instance. The ESR is computed as follows:

$$ESR = \frac{N_{\text{Evasion}}}{N_{\text{total}}}, \quad (1)$$

where N_{Evasion} denotes the number of requests that bypassed inspection, and $N_{\text{total}} = 1500$ is the total number of transmitted requests.

- **Backend Execution Rate (BER):** The fraction of bypassed requests that successfully triggered execution on the backend web framework. We validate execution by examining parameter reconstruction in backend logs, tracing HTTP/2 stream handling, and observing application state changes. This ensures that observed bypass events reflect actual code execution rather than harmless parsing anomalies.
- **Concurrent Attack Evasion Rate (CAER):** The success rate when SPCA applies all three topologies to each test case and considers an attack successful if any structure achieves bypass. The strategy models a topology-diversified attacker who selects the most effective structure per case.

$$CAER = \frac{|\{p \in P \mid \exists t \in T \text{ s.t. } \text{bypass}(p, t) = \text{true}\}|}{|P|}. \quad (2)$$

5.2 Overall Result

The overall experimental results are summarized in Table 1, while detailed outcomes for each tree configuration can be found in the Appendix: Table A5 for k-ary trees, Table A6 for skewed trees, and Table A7 for unbalanced trees. Our overall results are summarized as follows:

- Each tree topology of SPCA showed substantially higher ESR than the baseline (12.37%) using priority manipulation only, achieving 44.62% (k-ary), 49.66% (skewed), and 46.38% (unbalanced), as summarized in Table 1.

Table 1: Evasion Success Rates per WAF under HTTP/2 Tree Structures and Concurrent Attack Results

Type	WAF	k-ary tree	skewed tree	unbalanced tree	Baseline (PM Only)	CAER	BER
		(100 x 5 reqs)	(100 x 5 reqs)	(100 x 5 reqs)	(100 x 5 reqs)	(100 Malicious Test Case)	(20 WebApps)
Commercial	Microsoft Azure	212 (42.4%)	238 (47.6%)	228 (45.6%)	34 (6.8%)	95/100	13/20
	Amazon	183 (36.6%)	196 (39.2%)	192 (38.4%)	28 (5.6%)	76/100	8/20
	Google Cloud	229 (45.8%)	259 (51.8%)	240 (48.0%)	21 (4.2%)	86/100	14/20
	Cloudflare	235 (47.0%)	257 (51.4%)	234 (46.8%)	73 (14.4%)	88/100	13/20
	Safeline	241 (48.2%)	257 (51.4%)	232 (46.4%)	67 (13.4%)	98/100	16/20
	Fortinet	250 (50.0%)	268 (53.6%)	246 (49.2%)	68 (13.6%)	90/100	16/20
	Barracuda	196 (39.2%)	236 (47.2%)	216 (43.2%)	48 (9.6%)	91/100	16/20
ESR average		220.9 (44.18%)	242.0 (48.40%)	224.7 (44.94%)	48.4 (9.68%)	Avg : 89.14	Avg : 13.71
Open Source	ModSecurity	216 (43.2%)	251 (50.2%)	252 (50.4%)	85 (17.0%)	93/100	11/20
	Naxsi	231 (46.2%)	261 (52.2%)	243 (48.6%)	86 (17.2%)	93/100	15/20
	OpenWAF	215 (43.0%)	245 (49.0%)	227 (45.4%)	76 (15.2%)	83/100	13/20
	Janusec	204 (40.8%)	241 (48.2%)	224 (44.8%)	72 (14.4%)	90/100	11/20
	WAFbrain	251 (50.2%)	273 (54.6%)	244 (48.8%)	64 (12.8%)	91/100	20/20
	HiHTTPs	235 (47.0%)	257 (51.4%)	245 (49.0%)	82 (16.4%)	88/100	15/20
ESR average		225.33 (45.07%)	254.67 (50.93%)	239.17 (47.83%)	77.5 (15.5%)	Avg : 89.67	Avg : 14.17

- Concurrent attacks achieved a CAER of 89%, which is substantially higher than the evasion rates of any single-tree approach.
- Across 20 RFC-compliant HTTP/2 web frameworks, we found that, on average, 13.94 frameworks successfully executed the attack out of 100 malicious test cases against a single WAF. It implies that, despite structural variations introduced by stream dependency tree manipulation, SPCA can still be executed on a majority of real-world web frameworks.
- The web frameworks exhibited heterogeneous behaviors even for identically structured and WAF-bypassed SPCA requests. While some platforms (e.g., Laravel behind Nginx with PHP-FPM) successfully reassembled requests, others (e.g., Flask, Express) failed due to incomplete HTTP/2 stream handling.

SPCA consistently outperforms priority-manipulation-only scheduling across all WAFs. Our evaluation demonstrates that manipulating HTTP/2 stream dependency structures achieves significant bypass rates across all tested WAF implementations. As shown in Table 1, across all three tree configurations, SPCA consistently achieved significantly higher evasion rates—44.62% (k-ary), 49.66% (skewed), and 46.38% (unbalanced)—than 12.37% from the baseline (priority manipulation only).

Concurrent attack evasion reveals expanded protocol-level blind spots. To reflect practical attacker behavior, we adopt Concurrent Attack Evaluation Rate (CAER) in which a payload is considered successfully evaded if it bypasses the WAF under any of the three tested stream dependency structures: k-ary, skewed, or unbalanced trees. CAER captures evasion opportunities that would be missed under single topology analysis and reflects the realistic scenario where attackers concurrently retry structurally varied requests. As shown in the CAER column of Table 1, all evaluated WAFs—both commercial and open-source—exhibit substantially higher bypass rates when the three tree topologies are considered. For example, Microsoft Azure achieves per-tree ESR of 42.4%, 47.6%, and 45.6% under K-ary, Skewed, and Unbalanced trees, respectively.

5.3 Findings

Finding 1: Complementary Evasions Across Topologies. As shown in Table 1, the overall CAERs substantially exceed the evasion rate of any single topology. To explain this gain, for each of the 100 malicious test cases, we analyze the set of topologies under which evasion occurs and report the observed pairwise overlaps alongside the expected overlaps under independence. As shown in Table A3 in Appendix, the results suggest that the topologies exercise complementary behaviors. In particular, the unbalanced tree exhibits no overlap for Amazon and Fortinet, while the k-ary tree does so for Google Cloud Armor. Consequently, maximizing the overall evasion rate requires concurrent attacks, which are more likely to bypass diverse WAF detection mechanisms.

Finding 2: Backend Applications Exhibit Inconsistent Execution Results for Bypassed Requests. We analyzed how each HTTP/2-compliant framework parsed and handled the same SPCA-generated request. The results reveal a lack of consistency in request reassembly and parameter extraction across backends.

In Express (Node.js), although the request was received via the `http2` module, Express extracted query parameters solely from the `HEADERS` frame. For example, as shown in Listing 1 in the Appendix, this example underscores a critical failure mode: although the SPCA request was syntactically valid and successfully bypassed WAF inspection, Express failed to reassemble the logical input due to its reliance on `HEADERS`-only parsing for `GET` parameters. It reveals that even when detection mechanisms are evaded, incomplete reassembly at the application layer can suppress payload execution.

In Laravel, when executed using the default built-in server, the same request led to a local error page, often due to internal route handling or template rendering failures triggered by missing or malformed parameters. However, such an error does not necessarily imply that evasion failed; rather, it indicates that the malicious test case was accepted by the application and processed up to a point of failure within the backend logic. This distinction is important: even though the application returned an error, the injected request successfully bypassed the WAF and reached the backend request

handler, as shown in Listing 2 in the Appendix (Listing 2). Thus, we conservatively treat such local errors as evidence of backend execution, not as outright failure of the evasion attempt. This highlights the importance of structured execution tracing in our methodology.

Finding 3: RFC-Compliant Structures Were Incorrectly Blocked by Commercial WAFs. All stream topologies and transmission methods employed by SPCA strictly conformed to RFC 9113 [39]. However, only 3 out of 13 WAFs interpreted all valid requests correctly, while other WAFs misinterpreted some valid requests as malformed and either blocked them, or returned parsing errors. Furthermore, in configurations with deep dependency chains or wide sibling sets, 3 out of 10 WAFs (Microsoft Azure WAF, Amazon WAF, Fortinet WAF) failed to preserve priority semantics or forcibly dropped entire stream groups. These observations reveal a design gap: while HTTP/2 semantics permit flexible stream construction and frame scheduling, many WAFs continue to rely on rigid parsing models inherited from HTTP/1.1. Rather than implementing protocol-aware inspection, such WAFs enforce static assumptions about frame ordering and request layout.

6 DISCUSSION

6.1 Ethical Consideration

All experiments were conducted following established best practices for ethical security research and were confined to isolated environments fully under our administrative control. Experiments involving open-source WAFs were performed exclusively in local testbeds. For commercial WAFs, we generated only small-scale controlled traffic targeting our own domains, in strict accordance with each vendor’s vulnerability disclosure. We further engaged in coordinated responsible disclosure with all 13 evaluated vendors. Detailed disclosure status is provided in Appendix A.1.

6.2 Mitigation

Cloud-based intermediaries such as AWS ELB, Cloudflare Workers, or Nginx reverse proxies often mitigate protocol parsing discrepancies by normalizing HTTP/2 streams into HTTP/1.1-style payloads before forwarding them to WAFs or backend applications. While normalization can reduce some parsing inconsistencies, it sacrifices essential HTTP/2 features and is not a comprehensive mitigation. **Full Reassembly and RFC-Conformant Inspection.** A sustainable countermeasure is to defer request inspection until all HEADERS and DATA frames of a logical request have been reassembled, while adopting stream-level models aligned with RFC 9113 semantics. Although resource-intensive under high concurrency, this approach remains effective against fragmentation-based evasions by ensuring complete request reassembly.

Protocol Anomaly Detection. Machine learning models [3, 35] trained on benign HTTP/2 traffic can detect statistical deviations in stream structure, priority scheduling, and frame dispatch timing. While this approach requires sufficient training data and tuning, it complements signature-based defenses with adaptive anomaly detection capabilities.

7 RELATED WORK

Prior research on Web Application Firewall (WAF) [1, 10, 11, 41, 43] evasion spans multiple domains, including content-level mutation,

parser misalignment analysis, grammar-based fuzzing, and HTTP/2 stream manipulation.

Content-Level Evasion and Payload Mutation. Traditional WAF evasion techniques [33] primarily focus on modifying request payloads to circumvent pattern-matching filters. Tools such as WhatWaf [12], WAFNinja [18], and X-WAF [2] employ encoding, obfuscation, and transformation techniques to alter attack inputs. Recent approaches leverage adversarial mutation [41], reinforcement learning [3, 13, 31], and optimization-based transformations [10] to generate evasive variants. These techniques are generally effective against shallow rule sets but remain tailored to HTTP/1.1 parsing assumptions, overlooking structural features introduced in HTTP/2 such as stream multiplexing and fragmentation.

Parser Misalignment and Interpretation Divergence. Parsing inconsistencies between WAFs and backend frameworks have been widely recognized as a cause of inconsistent request interpretation [1, 37]. Notable examples include HTTP Parameter Pollution [5], ambiguity in Host header resolution [9], and misaligned parsing in multi-hop proxy or CDN chains [8]. Parsing discrepancies between WAFs and backend frameworks have long been recognized as a source of inconsistent request interpretation [5, 8, 9]. However, these studies primarily focus on HTTP/1.1 behavior and overlook the stream-level segmentation and dependency scheduling introduced in HTTP/2.

Differential Testing and Grammar-Guided Fuzzing. Grammar-based fuzzers [30, 36] such as T-Reqs [16], REQSMINER [44], and Gudifu [15] generate structured tests to expose discrepancies across middleware systems. Similarly, Geneva [32] and CenFuzz [14] apply syntax-driven transformations to discover censorship evasions. These tools reveal intermediary inconsistencies but fail to model HTTP/2 stream reassembly or validate backend execution.

8 CONCLUSION

In this paper, we introduced the Stream Parser Confusion Attack (SPCA), a novel protocol-level evasion technique that exploits structural discrepancies in HTTP/2 parsing between Web Application Firewalls (WAFs) and backend servers. SPCA uses RFC-compliant stream dependency trees and priority scheduling to decouple HEADERS and DATA frames across disjoint streams, enabling evasions without any payload mutation. Our evaluation across 13 real-world WAFs and 20 backend web frameworks combinations reveals 9,160 successful evasions and confirms backend execution, demonstrating a bypass rate exceeding 89% under concurrent stream tree configurations. SPCA exposes critical blind spots in modern WAFs, showing that structure-only manipulations can bypass detection at scale. We hope this work encourages future WAF designs to incorporate structure-aware parsing and full-stream correlation to bridge the protocol-level gap in HTTP/2 traffic inspection.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant (RS-2025-00563143, RS-2021-NR060143) and Institute of Information & communications Technology Planning & Evaluation (IITP) grant (No.2022-0-00411, IITP-2025-RS-2021-II211810) funded by the Korea government(MSIT).

References

- [1] Seyed Ali Akhavan, Bahruz Jabiyev, Ben Kallus, Cem Topcuoglu, Sergey Bratus, and Engin Kirda. 2025. WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls. *arXiv preprint arXiv:2503.10846* (2025).
- [2] Alemaktra. 2018. X-WAF. <https://github.com/Alemaktra/xWAF> (Accessed: 05.02.2025).
- [3] Dennis Appelt, Cu D Nguyen, Annibale Panichella, and Lionel C Briand. 2018. A machine-learning-driven evolutionary approach for testing web application firewalls. *IEEE Transactions on Reliability* 67, 3 (2018), 733–757.
- [4] Elias Athanasopoulos, Vasileios P Kemerlis, Michalis Polychronakis, and Evangelos P Markatos. 2012. ARC: protecting against HTTP parameter pollution attacks using application request caches. In *Applied Cryptography and Network SECURITY: 10th International Conference, ACNS 2012, Singapore, June 26–29, 2012. Proceedings 10*. Springer, 400–417.
- [5] Marco Balduzzi, Cristian T Gimenez, Davide Balzarotti, and Engin Kirda. 2011. Automated discovery of parameter pollution vulnerabilities in web applications. In *NDSS*.
- [6] David Belson. 2024. *Cloudflare 2024 Year in Review*. <https://blog.cloudflare.com/radar-2024-year-in-review/> (Accessed: 04.17.2025).
- [7] Cory Benfield. 2015. *python-hyper h2 library*. <https://python-hyper.org/en/latest/contributing.html> (Accessed: 05.16.2025).
- [8] Jing Chen, Vern Paxson, and Jian Jiang. 2020. Composition Kills: A Case Study of Email Sender Authentication. In *USENIX Security*. 2183–2199.
- [9] Jing et al. Chen. 2016. Host of Troubles: Multiple Host Ambiguities in HTTP Implementations. In *CCS*. 1516–1527.
- [10] B. R. Dawadi, B. Adhikari, and D. K. Srivastava. 2023. Deep Learning Technique-Enabled Web Application Firewall for the Detection of Web Attacks. *Sensors* 23, 4 (2023), 2073.
- [11] Luca Demetrio, Andrea Valenza, Gabriele Costa, and Giovanni Lagorio. 2020. Waf-a-mole: evading web application firewalls through adversarial machine learning. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing*. 1745–1752.
- [12] Eugene Kang (Ekultek). 2017. *WhatWaf*. <https://github.com/Ekultek/WhatWaf> (Accessed: 05.11.2025).
- [13] Mojtaba Hemmati and Mohammad Ali Hadavi. 2021. Using deep reinforcement learning to evade web application firewalls. In *2021 18th International ISC Conference on Information Security and Cryptology (ISCISC)*. IEEE, 35–41.
- [14] Qi-Xian Huang, Min-Yi Chiu, Ying-Feng Chen, and Hung-Min Sun. 2022. Attacking Websites: Detecting and Preventing HTTP Request Smuggling Attacks. *Security and Communication Networks* 2022, 1 (2022), 3121177.
- [15] Bahruz Jabiyev, Anthony Gavazzi, Kaan Onarlioglu, and Engin Kirda. 2024. Gudifu: Guided Differential Fuzzing for HTTP Request Parsing Discrepancies. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. 235–247.
- [16] Bahruz Jabiyev, Sebastian Sprecher, Kaan Onarlioglu, and Engin Kirda. 2021. T-Req: HTTP request smuggling with differential fuzzing. In *CCS*. 1805–1820.
- [17] James Kettle. 2021. HTTP/2: The Sequel is Always Worse. In *PortSwigger Research*.
- [18] khalilbijjou. 2016. *WAFNinja*. <https://github.com/khalilbijjou/WAFNinja> (Accessed: 05.10.2025).
- [19] Choongin Lee, Isa Jafarov, Sven Dietrich, and Heejo Lee. 2024. PRETT2: Discovering HTTP/2 DoS Vulnerabilities via Protocol Reverse Engineering. In *European Symposium on Research in Computer Security*. Springer, 3–23.
- [20] Emil Lerner. 2021. http2smugl: HTTP/2 Request Smuggling Security Testing Tool. In *GitHub Repository*.
- [21] Yingbo Li, Zhensong Huai, Xiaolong Yan, and Jing Liu. 2023. Request Smuggling Via HTTP/2 Cleartext in the Wild: Empirical Testing with Differential Fuzzing. In *2023 11th International Conference on Information Technology: IoT and Smart City (IToTSC)*. IEEE, 203–206.
- [22] Robin Marx. 2024. *HTTP*. <https://almanac.httparchive.org/en/2024/http> (Accessed: 05.10.2025).
- [23] Patrick Meenan. 2019. *Better HTTP/2 Prioritization for a Faster Web*. <https://blog.cloudflare.com/better-http-2-prioritization-for-a-faster-web/> (Accessed: 04.12.2025).
- [24] Liang Ming, Yu-Beng Leau, and Ying Xie. 2024. Distributed denial of service attack in HTTP/2: review on security issues and future challenges. *IEEE Access* (2024).
- [25] N. Moshe. 2022. *JS-ON: Security-OFF: Abusing JSON-Based SQL to Bypass WAF*. <https://kr.claroty.com/team82/research/js-on-security-off-abusing-json-based-sql-to-bypass-waf> (Accessed: 04.18.2025).
- [26] Ossinsight. 2025. *Web Framework - Ranking*. <https://ossinsight.io/collections/web-framework/> (Accessed: 04.12.2025).
- [27] OWASP. 2025. *OWASP coreruleset*. <https://coreruleset.org> (Accessed: 05.12.2025).
- [28] Lucas Pardue and David Belson. 2023. *Examining HTTP/3 usage one year on*. <https://blog.cloudflare.com/http3-usage-one-year-on> (Accessed: 05.11.2025).
- [29] Lucas Pardue and Julien Desgats. 2023. *HTTP/2 Rapid Reset: deconstructing the record-breaking attack*. <https://blog.cloudflare.com/technical-breakdown-http2-rapid-reset-ddos-attack/> (Accessed: 04.25.2025).
- [30] Shisong Qin, Fan Hu, Zheyu Ma, Bodong Zhao, Tingting Yin, and Chao Zhang. 2023. Nsfuzz: Towards efficient and state-aware network service fuzzing. *ACM Transactions on Software Engineering and Methodology* 32, 6 (2023), 1–26.
- [31] Zhenqing Qu, Xiang Ling, Ting Wang, Xiang Chen, Shouling Ji, and Chunming Wu. 2024. AdvSQLi: Generating Adversarial SQL Injections against Real-world WAF-as-a-service. *IEEE Transactions on Information Forensics and Security* 19 (2024), 2623–2638.
- [32] Zhenyu Qu, Xiaoyang Ling, and Chao Wu. 2022. AutoSpear: Towards automatically bypassing and inspecting web application firewalls. In *BlackHat Asia*.
- [33] Ivan Ristic. 2012. Protocol-level evasion of web application firewalls. *Black Hat USA* (2012).
- [34] B. Rozenfeld. 2024. *Best WAF Solutions in 2024–2025: Real-World Comparison*. <https://www.openappsec.io/post/best-waf-solutions-in-2024-2025-real-world-comparison> (Accessed: 05.04.2025).
- [35] Christian Scano, Giuseppe Floris, Biagio Montaruli, Luca Demetrio, Andrea Valenza, Luca Compagna, Davide Ariu, Luca Piras, Davide Balzarotti, and Battista Biggio. 2024. ModSec-Learn: Boosting ModSecurity with Machine Learning. In *International Symposium on Distributed Computing and Artificial Intelligence*. Springer, 23–33.
- [36] Sergej Schumilo, Cornelius Aschermann, Andrea Jemmett, Ali Abbasi, and Thorsten Holz. 2022. Nyx-net: network fuzzing with incremental snapshots. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 166–180.
- [37] Kaiwen Shen, Jianyu Lu, Yaru Yang, Jianjun Chen, Mingming Zhang, Haixin Duan, Jia Zhang, and Xiaofeng Zheng. 2022. Hdif: A semi-automatic framework for discovering semantic gap attack in http implementations. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–13.
- [38] Meenakshi Suresh, PP Amritha, Ashok Kumar Mohan, and V Anil Kumar. 2018. An investigation on HTTP/2 security. *Journal of Cyber Security and Mobility* (2018), 161–180.
- [39] Martin Thomson and Cory Benfield. 2022. HTTP/2. RFC 9113. doi:10.17487/RFC9113
- [40] W3Tech. 2025. *Usage statistics of HTTP/2 for websites*. <https://w3techs.com/technologies/details/ce-http2> (Accessed: 04.10.2025).
- [41] Qi Wang, Jianjun Chen, Zheyu Jiang, Run Guo, Ximeng Liu, Chao Zhang, and Haixin Duan. 2024. Break the wall from bottom: Automated discovery of protocol-level evasion vulnerabilities in web application firewalls. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 185–202.
- [42] Maarten Wijnants, Robin Marx, Peter Quax, and Wim Lamotte. 2018. HTTP/2 prioritization and its impact on web performance. In *Proceedings of the 2018 World Wide Web Conference*. 1755–1764.
- [43] Cong Wu, Jing Chen, Simeng Zhu, Wenqi Feng, Kun He, Ruiying Du, and Yang Xiang. 2024. Wafbooster: Automatic boosting of waf security against mutated malicious payloads. *IEEE Transactions on Dependable and Secure Computing* (2024).
- [44] Linkai Zheng, Xiang Li, Chuhan Wang, Run Guo, Haixin Duan, Jianjun Chen, Chao Zhang, and Kaiwen Shen. 2024. REQSMINER: Automated Discovery of CDN Forwarding Request Inconsistencies and DoS Attacks with Grammar-based Fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium, NDSS*, Vol. 24.

Listing 2: Laravel Log Showing SQL Exception Triggered by SPCA Payload

```
[2025-05-01 10:14:02] local.ERROR: SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''1'-- OR 1=1'' at line 1 (SQL: SELECT * FROM users WHERE id='1'-- OR 1=1)
```

A APPENDIX

A.1 Detailed Disclosure

We have disclosed our experimental findings to all 13 evaluated vendors (since August 19, 2025), including the relevant open-source projects, following best practices of ethical security research. All experiments were conducted in isolated environments fully under our administrative control. For open-source WAFs, all testing was performed locally, ensuring that no third-party systems were affected. For commercial WAFs, we generated only small-scale controlled traffic targeting our own domains, in strict accordance with bug bounty or vulnerability reporting policies. We further engaged in coordinated responsible disclosure by sharing reproduction scripts, stream-priority configurations, and a safe local test harness.

AWS WAF. We reported our findings to AWS WAF via HackerOne, where the submission was accepted and evaluated by the AWS CNA team. Following their assessment, AWS determined that no CVE would be issued for this report, as the identified behavior was remediated server-side and did not require customer action, placing it outside AWS’s CVE publication scope. AWS confirmed that the report contains valid security findings and that the underlying issue has been addressed on the AWS side. Their security team stated that they are exploring alternative mechanisms to communicate the fix to customers, such as a security bulletin or blog-style advisory, and noted that some of these mechanisms include publicly giving credit to us. We provided reproduction scripts and additional supporting materials upon request to facilitate validation and communication.

ModSecurity (OWASP CRS). We provided Markdown-based reproduction instructions and PoC scripts. The maintainers responded with error logs from their setup, showing SQLi detection under one configuration. We are now working to align CRS ruleset versions and backend environments to resolve the discrepancy. The exchange has been collaborative and detail-oriented.

Google Cloud Armor. The report was formally logged under their internal issue tracker, assigned a case number, and escalated in priority. Their engineering team informed us that the issue is in active triage, and they are reviewing our reproduction steps.

Cloudflare. Their security team acknowledged the report and confirmed that the evasion technique was understood, noting that the finding would inform future product updates.

Fortinet PSIRT. Acknowledged receipt and opened an internal investigation. The case has been assigned and validation is in progress. **Others.** We have notified the other relevant WAF vendors via their responsible disclosure channels and await their triage feedback.

Listing 1: Express Log Showing Parameter Parsing Failure due to Missing Reassembly

```
[2025-05-01T10:16:47.281Z] Opened stream (id=3); awaiting HEADERS for /search
[2025-05-01T10:16:49.282Z] HEADERS frame received: :method=GET, :path=/search (stream=3)
[2025-05-01T10:16:52.283Z] DATA frame received: payload="id='1'-- OR 1=1"
[2025-05-01T10:16:53.284Z] Parsed query parameters: {}
[2025-05-01T10:16:54.284Z] Received ID: undefined
[2025-05-01T10:16:55.285Z] Warning: Expected parameter "id" missing from request
```

Table A3: Evasion Success under Commercial WAF Concurrent Attack of 100 Malicious Payloads

WAF	K^*	S^{**}	U^{***}	$K \cap S$	$S \cap U$	$U \cap K$	$K \cap S \cap U$	Fail
Azure	29	29	23	6	2	5	1	5
Amazon	19	21	24	2	3	2	2	27
Google Cloud	26	27	27	–	6	–	–	14
Cloudflare	22	24	34	5	2	1	0	12
Safeline	25	24	37	5	3	4	0	2
Fortinet	28	28	28	6	–	–	–	10
Barracuda	26	30	21	3	7	4	0	9

* : k-ary tree, ** : skewed tree, ***: unbalanced tree

Table A4: Evasion Success in Case 1 (uniform priority) and Case 2 (priority manipulation). The numbers in Case 2 correspond to the number of successful evasions observed among the 500 malicious requests.

Type	WAF ^s	Evasion Success Rate	
		Case 1	Case 2
Commercial	Microsoft Azure WAF	5/100	34/500
	Amazon WAF	2/100	28/500
	Google Cloud Armor	5/100	21/500
	Cloudflare WAF	24/100	73/500
	Safeline WAF	22/100	67/500
	Fortinet WAF	13/100	68/500
	Barracuda WAF	11/100	48/500
Open Source	ModSecurity	27/100	85/500
	Naxsi	24/100	86/500
	OpenWAF	23/100	76/500
	Janusec	21/100	72/500
	WAFbrain	18/100	64/500
	HiHTTPs	22/100	82/500

Table A1: Priority Manipulation without Stream Dependency Manipulation in ModSecurity WAF

Test Cases	Priority Group	Assigned Streams	Weight Range	#Bypassed / #Requests	Evasion Rate (%)
Case 1 : Uniform Priority	Uniform	1–20	Uniform (1–256)	13 / 100	13.0%
Case 2 : Priority Manipulation	Lowest	1–4	100–120	9 / 100	9.0%
	Low	5–8	130–150	14 / 100	14.0%
	Normal	9–12	160–190	17 / 100	17.0%
	High	13–16	200–220	22 / 100	22.0%
	Highest	17–20	230–256	23 / 100	23.0%

Table A2: Top 20 HTTP/2-Compatible Web Frameworks by OSSInsight GitHub Stars (2025)

Rank	Framework	Language	Version	GitHub Stars
1	Laravel	PHP	12.0.1	90.8k
2	Django	Python	5.2.1	90.78k
3	FastAPI	Python	0.115.12	86.29k
4	Gin	Go	1.9.0	86.77k
5	Spring Boot	Java	3.5.0	84.01k
6	Flask	Python	3.1.1	76.6k
7	NestJS	TypeScript	11.1.2	73.36k
8	Express	Node.js	5.1.0	72.85k
9	Strapi	JavaScript	4.15.0	69.54k
10	Rails	Ruby	7.1.0	58.3k
11	ASP.NET Core	.NET	8.0	39.55k
12	Koa	Node.js	2.14.1	39.06k
13	Beego	Go	2.0.1	36.24k
14	Fastify	Node.js	4.11.0	35.01k
15	Echo	Go	4.10.0	33.15k
16	Symfony	PHP	6.4.22	32.59k
17	Rocket	Rust	0.5.0-rc2	27.06k
18	Actix Web	Rust	4.0.0	23.9k
19	Phoenix	Elixir	1.7.0	23.79k
20	AdonisJS	JavaScript	6.0.0	18.66k

Table A5: Evasion results by priority level using k-ary tree topology

Type	WAF	Lowest	Low	Normal	High	Highest
Commercial	Microsoft Azure WAF	31	38	41	48	54
	Amazon WAF	42	47	45	45	44
	Google Cloud Armor	35	40	47	50	57
	Cloudflare WAF	35	42	49	50	59
	Safeline WAF	37	45	50	52	57
	Fortinet WAF	39	44	51	56	60
	Barracuda WAF	30	35	39	44	48
Open Source	ModSecurity	32	39	44	48	53
	Naxsi	36	41	47	51	56
	OpenWAF	33	39	42	46	52
	Janusec	31	36	41	45	51
	WAFbrain	39	45	51	56	60
	HiHTTTPs	37	42	48	51	57
Commercial	Microsoft Azure WAF	1/20	2/20	2/20	2/20	4/20
	Amazon WAF	3/20	2/20	4/20	2/20	2/20
	Google Cloud Armor	2/20	2/20	2/20	2/20	4/20
	Cloudflare WAF	2/20	2/20	2/20	2/20	4/20
	Safeline WAF	2/20	2/20	2/20	2/20	3/20
	Fortinet WAF	3/20	3/20	3/20	3/20	4/20
	Barracuda WAF	1/20	1/20	1/20	2/20	2/20
Open Source	ModSecurity	2/20	2/20	2/20	2/20	3/20
	Naxsi	3/20	3/20	3/20	3/20	4/20
	OpenWAF	2/20	2/20	2/20	2/20	3/20
	Janusec	1/20	2/20	2/20	2/20	3/20
	WAFbrain	4/20	4/20	4/20	4/20	4/20
	HiHTTTPs	3/20	3/20	3/20	3/20	4/20

Table A6: Evasion results by priority level using skewed tree topology

Type	WAF	Lowest	Low	Normal	High	Highest
Commercial	Microsoft Azure WAF	41	43	46	52	56
	Amazon WAF	41	47	42	43	50
	Google Cloud Armor	43	46	53	57	60
	Cloudflare WAF	45	45	53	57	57
	Safeline WAF	43	47	50	57	60
	Fortinet WAF	46	50	55	57	60
Open Source	Barracuda WAF	42	45	48	49	52
	ModSecurity	42	46	51	54	58
	Naxsi	46	50	52	54	59
	OpenWAF	43	45	48	52	57
	Janusec	42	45	48	50	56
	WAFbrain	45	48	54	56	60
Commercial	HiHTTTPs	44	47	52	55	59
	Microsoft Azure WAF	2/20	2/20	2/20	3/20	4/20
	Amazon WAF	3/20	1/20	2/20	2/20	2/20
	Google Cloud Armor	2/20	2/20	3/20	3/20	4/20
	Cloudflare WAF	2/20	2/20	3/20	3/20	3/20
	Safeline WAF	2/20	2/20	2/20	3/20	3/20
	Fortinet WAF	3/20	3/20	3/20	3/20	4/20
Open Source	Barracuda WAF	2/20	2/20	2/20	2/20	2/20
	ModSecurity	2/20	2/20	2/20	2/20	3/20
	Naxsi	3/20	3/20	3/20	3/20	4/20
	OpenWAF	2/20	2/20	2/20	2/20	3/20
	Janusec	2/20	2/20	2/20	2/20	3/20
	WAFbrain	4/20	4/20	4/20	4/20	4/20
Open Source	HiHTTTPs	3/20	3/20	3/20	3/20	4/20

Table A7: Evasion results by priority level using unbalanced tree topology

Type	WAF	Lowest	Low	Normal	High	Highest
Commercial	Microsoft Azure WAF	40	42	45	50	51
	Amazon WAF	44	49	43	46	50
	Google Cloud Armor	42	45	48	51	54
	Cloudflare WAF	41	44	47	50	52
	Safeline WAF	41	44	46	50	51
	Fortinet WAF	43	46	49	52	56
Open Source	Barracuda WAF	40	42	45	47	52
	ModSecurity	44	47	51	54	56
	Naxsi	43	46	49	52	53
	OpenWAF	42	44	46	48	51
	Janusec	41	44	46	48	50
	WAFbrain	43	46	49	52	55
Commercial	HiHTTTPs	43	46	49	52	55
	Microsoft Azure WAF	2/20	2/20	2/20	3/20	4/20
	Amazon WAF	2/20	3/20	1/20	2/20	2/20
	Google Cloud Armor	2/20	2/20	3/20	3/20	4/20
	Cloudflare WAF	2/20	2/20	3/20	3/20	3/20
	Safeline WAF	2/20	2/20	2/20	3/20	3/20
	Fortinet WAF	3/20	3/20	3/20	3/20	4/20
Open Source	Barracuda WAF	2/20	2/20	2/20	2/20	2/20
	ModSecurity	2/20	2/20	2/20	2/20	3/20
	Naxsi	3/20	3/20	3/20	3/20	4/20
	OpenWAF	2/20	2/20	2/20	2/20	3/20
	Janusec	2/20	2/20	2/20	2/20	3/20
	WAFbrain	4/20	4/20	4/20	4/20	4/20
Open Source	HiHTTTPs	3/20	3/20	3/20	3/20	4/20